Student Journey CRM & BI - Project Report

An account of the design and build of an end-to-end CRM and business-intelligence system for an Ireland-based education and visa consultancy: how the data pipeline, relational model and Power BI dashboards were created, and the decisions made along the way.

Introduction and objectives

This project designs and builds an end-to-end CRM and business-intelligence system for an Ireland-based education and visa consultancy that recruits international students into Irish universities. The goal was to create a single, connected data model covering the entire recruitment lifecycle -from first enquiry through application, offer, visa, arrival and university commission -and to surface it through an interactive Power BI dashboard suite.

The objectives were to:

- generate and clean a realistic multi-table dataset representing the agency's operations;
- enforce referential integrity and business rules so the data behaves like a real CRM;
- model the data relationally in PostgreSQL;
- build a dashboard suite that measures lead conversion, admissions, visas, student cases and commission finance; and
- produce a business analysis report from the results.

The project demonstrates the full data-analytics lifecycle -data generation, engineering, modelling, visualisation and reporting -as one coherent pipeline rather than a set of isolated tasks.

Business problem and scope

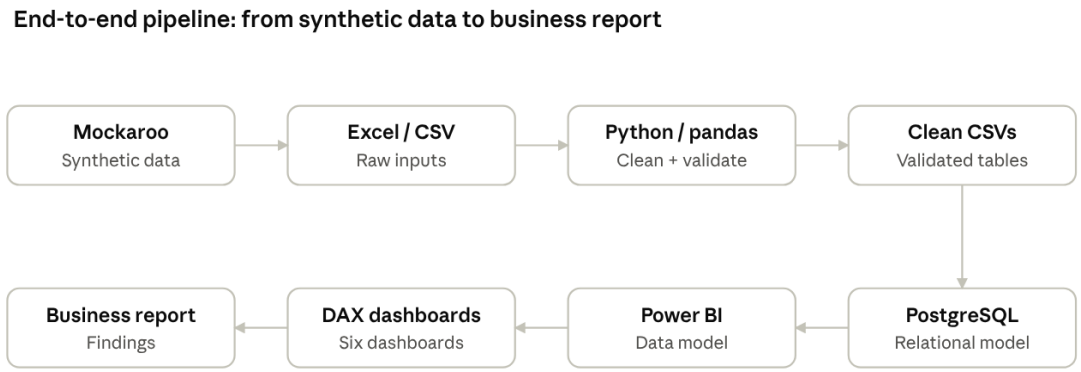
The agency earns revenue from commissions paid by partner universities when a recruited student enrols, not from fees charged to students. Because revenue is realised only at the end of a long journey, the central business problem is **student-journey leakage**: prospects enquire or attend consultations but drop out before applying, before an offer, before a visa, or before arriving.

The project is scoped to model and measure that journey end to end. In scope: lead and marketing data, consultant assignment, university applications, offers, visa cases, arrivals, and the commission each successful journey earns. Out of scope: detailed case-event or contact history, real university contracts, and full operating-cost accounting -these were excluded because the supporting data did not exist and would have required assumptions beyond the project's purpose.

The design principle throughout was to treat the whole thing as one connected chain -student → journey → application → visa → arrival → commission -so that every dashboard is a different view of the same underlying process.

System architecture and tools

The system is a conventional, layered analytics stack. Each layer has a single responsibility and a clean hand-off to the next, which keeps the pipeline reproducible and easy to debug.



system architecture · synthetic data to business report

Mockaroo generates the synthetic raw data; Python with pandas cleans, repairs and validates it in Jupyter; the validated tables are exported as CSVs and loaded into a PostgreSQL database (managed in pgAdmin); Power BI connects to PostgreSQL, builds the data model, and renders the dashboards with DAX measures; and the final layer is the written business analysis report.

Data sources and generation

All data is synthetic, generated in Mockaroo to resemble a real agency's operations and then deliberately made messy (mixed date formats, fake-null tokens, inconsistent labels) so the cleaning pipeline had realistic problems to solve. The project uses seven linked tables.

Table	Rows	Role
students	1,000	people recruited
consultants	5	staff handling cases
marketing_campaigns	30	paid acquisition
student_journeys	1,000	the lifecycle spine (one per lead)
university_applications	1,118	applications lodged
visa_applications	289	visa cases
university_commissions	135	revenue from successful journeys

The student population is set to Myanmar nationality to reflect the agency's target market, and all personal fields (names, emails, phone numbers) are fictional and safe to publish.

Data cleaning and preparation

Cleaning was built on four reusable helper functions so the same logic applied consistently across every table:

- `standardize_columns` -normalises column names to lower snake_case;

- `clean_text` -trims whitespace and converts fake-null tokens (`-`, `n/a`, `NULL`) to true missing values;
- `clean_id` -coerces IDs to clean strings (e.g. `5.0` → `5`);
- `parse_mixed_date` -handles both Excel serial numbers and string dates.

On top of these, table-specific repairs were applied: reversed campaign start/end dates were swapped, future lead dates were pulled back behind the snapshot, intake years were forced to be plausible relative to the lead date, lead sources and pipeline stages were mapped to a controlled vocabulary, and missing consultants were assigned.

The most involved step was **stage reconciliation**: rather than trust the recorded stage, the pipeline infers the furthest stage each student actually reached from evidence in the application and visa tables, and promotes the journey to that stage. This prevents inconsistencies such as a student holding an approved visa while still marked at an earlier stage.

Data validation and quality assurance

The pipeline ends with a block of assertion-based checks that must all pass before any table is saved -if one fails, the export is blocked. This treats data quality as a gate, not an afterthought.

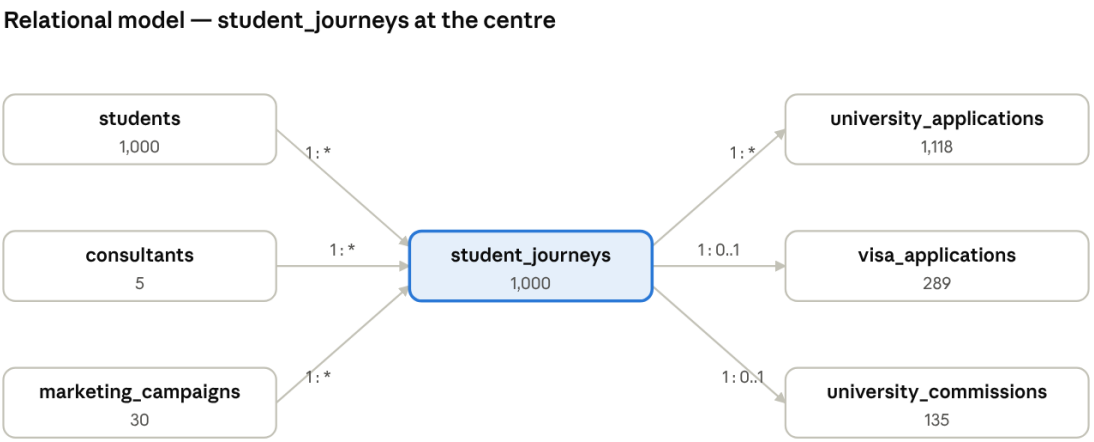
The checks cover four areas:

- **Primary keys** -no nulls and no duplicates in any key column.
- **Foreign keys** -every application, visa and commission resolves to a valid journey, and each visa/commission to a valid application.
- **Composite consistency** -each visa's `(app_id, sj_id)` pair matches its parent application exactly.
- **Business rules** -no future lead dates, no pre-application student holding an application, and intake years that are plausible relative to the lead date.

These checks confirmed the final dataset is internally consistent: clean keys, no orphan records, and relationships that hold across all seven tables.

Data modelling

The seven tables form a star-like model with `student_journeys` as the central spine. Students, consultants and marketing campaigns feed into it (each one-to-many), and applications, visas and commissions hang off it downstream.



relational model · seven tables around the student journey

Relationships use single-direction filtering. A journey has many applications but at most one visa and at most one commission -modelled as one-to-zero-or-one, since most journeys never reach those stages. A dedicated date table (DimDate) was added for time analysis, linked to the lead date. Keeping the cardinalities honest here is what stops the dashboards from showing phantom blanks later.

Dashboard design and key measures

The Power BI layer is organised as six dashboards, each a view of the same end-to-end process rather than an isolated report:

1. Executive Overview -the management cockpit (funnel and headline KPIs).
2. Lead & Marketing Analysis.
3. University Admission & Application Tracking.
4. Visa Analysis.
5. Student Case Tracking.
6. Finance & Operations.

Measures are written in DAX. Counts use `DISTINCTCOUNT` on the relevant key, and rates are built with `DIVIDE` to handle empty denominators safely. Representative examples:

```
Total Leads = DISTINCTCOUNT('student_journeys'[sj_id])

Offers Received =
CALCULATE(
    DISTINCTCOUNT('university_applications'[app_id]),
    'university_applications'[application_status] = "Offer Issued")

Visa Approval Rate =
DIVIDE([Approved Visas], [Approved Visas] + [Refused Visas])

Commission Collection Rate =
DIVIDE([Commission Collected], [Earned Commission])
```

Design choices worth noting: a separate date table drives time analysis; display measures convert blank outcomes into readable labels ("Pending / No Final Outcome"); and the executive financial pipeline is built from mutually exclusive commission statuses so the amounts do not double-count.

Key design decisions and challenges

- **Commission as the revenue model.** An early idea of charging students fees was rejected in favour of university commissions, which matches how the business actually earns. Commission is recognised only for approved-visa journeys, split into Earned (student arrived) and Forecast (visa approved, not yet arrived).
- **A realistic commission lifecycle.** Rather than a single "paid/unpaid" flag, commissions move through Forecast → Earned-not-invoiced → Invoiced → Paid/Overdue, with date-ordering rules enforced (invoice after earned, payment after invoice, nothing in the future).
- **Removing the case-events table.** A planned table of calls, emails and stage changes was dropped because no such data existed; this is an accepted limitation rather than fabricated data.
- **Keeping cardinalities honest.** Journeys relate to visas and commissions as one-to-zero-or-one, not one-to-one, which avoids blank rows leaking into slicers and visuals.
- **Not overclaiming profit.** Contribution after marketing is reported as exactly that -earned commission minus marketing spend -and is never labelled profit, because operating costs are not in the data.

The main technical challenge was reconciling inconsistent stages across three tables; the evidence-ranking approach in cleaning solved it deterministically.

Results and deliverables

The project produced a complete, working analytics system:

- seven cleaned, validated and internally consistent CSV datasets;
- a documented Python cleaning-and-validation pipeline across three notebooks;
- a relational PostgreSQL database implementing the model;
- a six-dashboard Power BI suite with DAX measures; and
- a business analysis report with findings and recommendations.

The analysis it supports quantifies the business clearly: a 6% lead-to-arrival conversion, a 90% visa approval rate, 67% applicant-level offer success, and €174k of earned commission at a 50.7% collection rate. The system turns a messy, multi-table dataset into a single coherent view of the recruitment pipeline -which was the project's objective.

Future work

- Reintroduce a case-events table with stage-change timestamps to enable time-in-stage and contact-history analysis -the step that would turn leakage *measurement* into leakage *diagnosis*.
- Add application and visa processing durations once dated events exist.
- Replace synthetic commission assumptions with real university agreements and payment reconciliation.

- Strengthen marketing attribution (a dedicated campaign flag) to support proper channel-ROI analysis.
- Build a dropout-risk model to flag journeys likely to leak before they do.
- Integrate with a live CRM (e.g. Salesforce) so the pipeline runs on real, refreshing data rather than a snapshot.

Conclusion

The project delivers a full data-analytics lifecycle as one connected system: synthetic data generated, cleaned and validated in Python, modelled in PostgreSQL, and visualised in a Power BI suite that answers real business questions about the recruitment pipeline. The engineering emphasis -reusable cleaning logic, assertion-based validation, and an honest relational model -is what makes the downstream analysis trustworthy.

Its limits are understood and documented: the data is synthetic and captures a single point in time, so the natural next step is real, event-level data that would let the system explain leakage rather than only measure it. As it stands, the project demonstrates the end-to-end skills of a working data analyst, from raw data to a decision-ready report.